

SQL — beginner se advanced tak, ISI project ke real tables pe

Har example Kapil ERP ke asli PostgreSQL tables use karta hai, taaki seekha hua seedha kaam aaye. Practice: `env/bin/python config/manage.py dbshell` (psql kholta hai).
Dhyan: dev DB pe SELECT karo jitna chaaho — INSERT/UPDATE kabhi haath se mat karo (single-writer rule yahan bhi lagti hai!).

Tables hum use karenge: `expense_workerledgerentry` (ledger) · `expense_addasettlement` · `expense_stageworkassignment` (SWA) · `production_workerstagetask` (WST) · `production_workerstagecontribution` (WSC) · `production_adda` · `raw_materials_clothroll` · `accounts_user`.

Level 1 — SELECT: data PADHNA

```
-- सबसे simple: poori table (chhoti tables pe hi karna!)
SELECT * FROM production_adda;

-- Sirf chaahiye wale columns – hamesha yehi aadat rakho
SELECT code, status, started_at FROM production_adda;

-- WHERE = filter. = , != , > , < , IN, LIKE
SELECT code, status FROM production_adda WHERE status = 'in_progress';
SELECT roll_id, cost_per_kg FROM raw_materials_clothroll
  WHERE cost_per_kg IS NULL;          -- NULL ka check IS NULL hota hai, = NULL
nahi!

-- ORDER BY + LIMIT = "latest 10"
SELECT reference, status, settled_at FROM expense_addasettlement
  ORDER BY id DESC LIMIT 10;
```

Hinglish samajh: SELECT = “dikhao”, FROM = “kahan se”, WHERE = “sirf woh jahan...”.
NULL matlab “pata nahi” — isliye `cost_per_kg IS NULL` wale rolls costing dashboard pe ‘un-priced’ banner banate hain (ADR-0009 honest-NULL).

Level 2 — JOIN: do tables ko jodna

FK column dusri table ki row ka pointer hai. JOIN us pointer ko follow karta hai.

```
-- Har ledger row ke saath worker ka email (INNER JOIN: dono side match zaroori)
SELECT w.email, l.entry_type, l.category, l.amount
  FROM expense_workerledgerentry l
  JOIN accounts_user w ON w.id = l.worker_id
 ORDER BY l.id;

-- LEFT JOIN: baayi table ki SAB rows, right side mile na mile (NULL aa jata hai)
-- "har Adda + uska latest settlement (ho to)"
SELECT a.code, s.reference, s.status
  FROM production_adda a
  LEFT JOIN expense_addasettlement s ON s.adda_id = a.id;
```

Project mein yehi hota hai: settlement detail page WSC → task → worker → stage 4-table JOIN hai — Django `select_related('task__worker', ...)` likhta hai, SQL mein wahi INNER JOIN chain ban jaati hai.

Level 3 — Aggregates: GROUP BY, SUM, COUNT

```
-- THE project query: worker ka live balance (pending payable)
SELECT worker_id,
       SUM(CASE WHEN entry_type='credit' THEN amount ELSE -amount END) AS payable
  FROM expense_workerledgerentry
 GROUP BY worker_id;

-- Har Adda ke kitne pieces report hue (production truth se)
SELECT a.code, SUM(c.reported_quantity) AS pieces
  FROM production_workerstagecontribution c
  JOIN production_workerstagetask t ON t.id = c.task_id
  JOIN production_addastagerecord sr ON sr.id = t.stage_record_id
  JOIN production_adda a ON a.id = sr.adda_id
 WHERE t.status IN ('completed', 'verified')
 GROUP BY a.code;

-- HAVING = group banne ke BAAD ka filter (WHERE row-level hai, HAVING group-level)
SELECT worker_id, COUNT(*) AS advances
  FROM expense_workeradvance GROUP BY worker_id HAVING COUNT(*) > 2;
```

Kyun yeh design: balance kabhi STORE nahi hota — hamesha yeh SUM chalta hai. Stored total drift karta; SUM jhooth nahi bolta (ledger_service docstring yehi kehta hai).

Level 4 — Subqueries & CASE

```
-- Workers jinka koi UNSETTLED completed kaam hai (subquery IN)
SELECT email FROM accounts_user
WHERE id IN (
  SELECT t.worker_id
    FROM production_workerstagecontribution c
   JOIN production_workerstagetask t ON t.id = c.task_id
  WHERE c.settlement_line_id IS NULL
        AND t.status IN ('completed','verified'));

-- CASE = SQL ka if/else: verified-else-reported (settlement EXACTLY yehi karta hai)
SELECT c.id,
       COALESCE(c.verified_quantity, c.reported_quantity) AS settle_qty
  FROM production_workerstagecontribution c;
-- COALESCE = pehla NON-NULL value. Yehi ek line V2 ka "verified-else-reported".
```

Level 5 — Window functions (advanced, par sona)

GROUP BY rows ko nigaal jaata hai; window functions har row RAKH ke saath-saath group-math dete hain.

```
-- Har ledger row ke saath us worker ka RUNNING balance
SELECT id, worker_id, entry_type, amount,
       SUM(CASE WHEN entry_type='credit' THEN amount ELSE -amount END)
       OVER (PARTITION BY worker_id ORDER BY id) AS running_balance
  FROM expense_workerledgerentry;

-- Har Adda ka LATEST settlement (ROW_NUMBER pattern – interview favourite)
SELECT * FROM (
  SELECT s.*, ROW_NUMBER() OVER (PARTITION BY adda_id ORDER BY id DESC) rn
    FROM expense_addasettlement s) x
 WHERE rn = 1;

-- Worker productivity rank per stage
SELECT t.worker_id, sr.id AS stage_record,
       SUM(c.reported_quantity) AS pieces,
       RANK() OVER (PARTITION BY sr.id ORDER BY SUM(c.reported_quantity) DESC)
  FROM production_workerstagecontribution c
   JOIN production_workerstagetask t ON t.id=c.task_id
   JOIN production_addastagerecord sr ON sr.id=t.stage_record_id
 GROUP BY t.worker_id, sr.id;
```

`PARTITION BY` = “kis group ke andar”, `ORDER BY` (window mein) = “kis tarteeb se chalta hua”.
G4/Reporting phase yehi patterns use karega.

Level 6 — EXPLAIN & indexes: query TEZ kyun/kab hoti hai

```
EXPLAIN ANALYZE
SELECT * FROM production_workerstagetask
  WHERE worker_id = 7 AND status = 'completed';
-- Dekho: "Index Scan using production_..._worker_status_idx" = FAST.
-- "Seq Scan" on a big table = poora table padha – index missing ya bekaar.
```

Index = sorted shortcut. Humne (worker, status) pe isliye banaya (model Meta indexes) kyunki dashboard yehi filter maarta hai. Rule: index un columns pe jo WHERE/JOIN/ORDER mein BAAR-BAAR aate hain; har column pe nahi (writes mehengi ho jaati hain).

Level 7 — Transactions & locks (SQL side of LEARNING/03)

```
BEGIN;                                -- transaction shuru
SELECT * FROM expense_addasettlement WHERE id = 3 FOR UPDATE;  -- row LOCK
-- ... checks + writes ...
COMMIT;                                -- sab ek saath; ya ROLLBACK = kuch nahi
hua
```

`FOR UPDATE` wahi hai jo Django `select_for_update()` banata hai. Lock ORDER fixed rakhna (advisory 5374 → ADST → SR → profile → advance) deadlock ka ilaaj hai. `SELECT pg_advisory_xact_lock(5374)` = naam-wala app-level lock.

Level 8 — Django ORM ↔ SQL (translation table)

ORM	SQL
<code>Adda.objects.filter(status='in_progress')</code>	<code>SELECT ... WHERE status='in_progress'</code>
<code>.select_related('product')</code>	<code>JOIN production_product ...</code>
<code>.prefetch_related('stage_records')</code>	2nd query + Python join (reverse FK)
<code>.aggregate(s=Sum('amount'))</code>	<code>SELECT SUM(amount)</code>
<code>.values('worker').annotate(n=Count('id'))</code>	<code>GROUP BY worker_id</code>
<code>.select_for_update()</code>	<code>FOR UPDATE</code>
<code>Q(a=1) Q(b=2)</code>	<code>WHERE a=1 OR b=2</code>
<code>F('advance_outstanding_before')</code>	column-vs-column compare (CHECK constraint mein use hua)
<code>__isnull=True</code>	<code>IS NULL</code>
<code>.exists()</code>	<code>SELECT 1 ... LIMIT 1</code> (poori row nahi laata — sasta)

Debug ka jaadu: `print(qs.query)` — kisi bhi queryset ka asli SQL dikha deta hai. Perf-base-line tests `assertNumQueries` se query COUNT pin karte hain — N+1 regression test fail karwa deta hai.

Level 9 — “Woh query kaise likhun?” — ek recipe

1. **Bolo plain Hindi/English mein:** “har worker ka unsettled expected paisa”.
2. **Tables pehchano:** WSC (paisa-line) + WST (worker, status).
3. **Filters likho:** task completed/verified · settlement_line IS NULL · expected_earning IS NOT NULL.
4. **Shape choose karo:** per-worker total ⇒ GROUP BY worker_id + SUM.
5. **Likho, EXPLAIN se check karo, edge cases poochho** (era-A excluded? voided? — domain rules SQL me bhi lagti hain; isi liye READS bhi payroll_service se guzarte hain, jo era rules jaanta hai).

```
SELECT t.worker_id, SUM(c.expected_earning) AS expected_unsettled
FROM production_workerstagecontribution c
JOIN production_workerstagetask t ON t.id = c.task_id
WHERE t.status IN ('completed', 'verified')
      AND c.expected_earning IS NOT NULL
      AND c.settlement_line_id IS NULL
GROUP BY t.worker_id;
```

(Production code isi ka era-aware version hai: `payroll_service.unsettled_expected`.)

Aage padhne ko

- LEARNING/03 (locks ki Django side) · LEARNING/02 (rishtey) · [postgresql.org/docs/current/tutorial.html](https://www.postgresql.org/docs/current/tutorial.html) (official, बहुत अच्छा hai)
- Golden rule yaad rakhna: **app ke through likho, SQL se sirf PADHO.**